

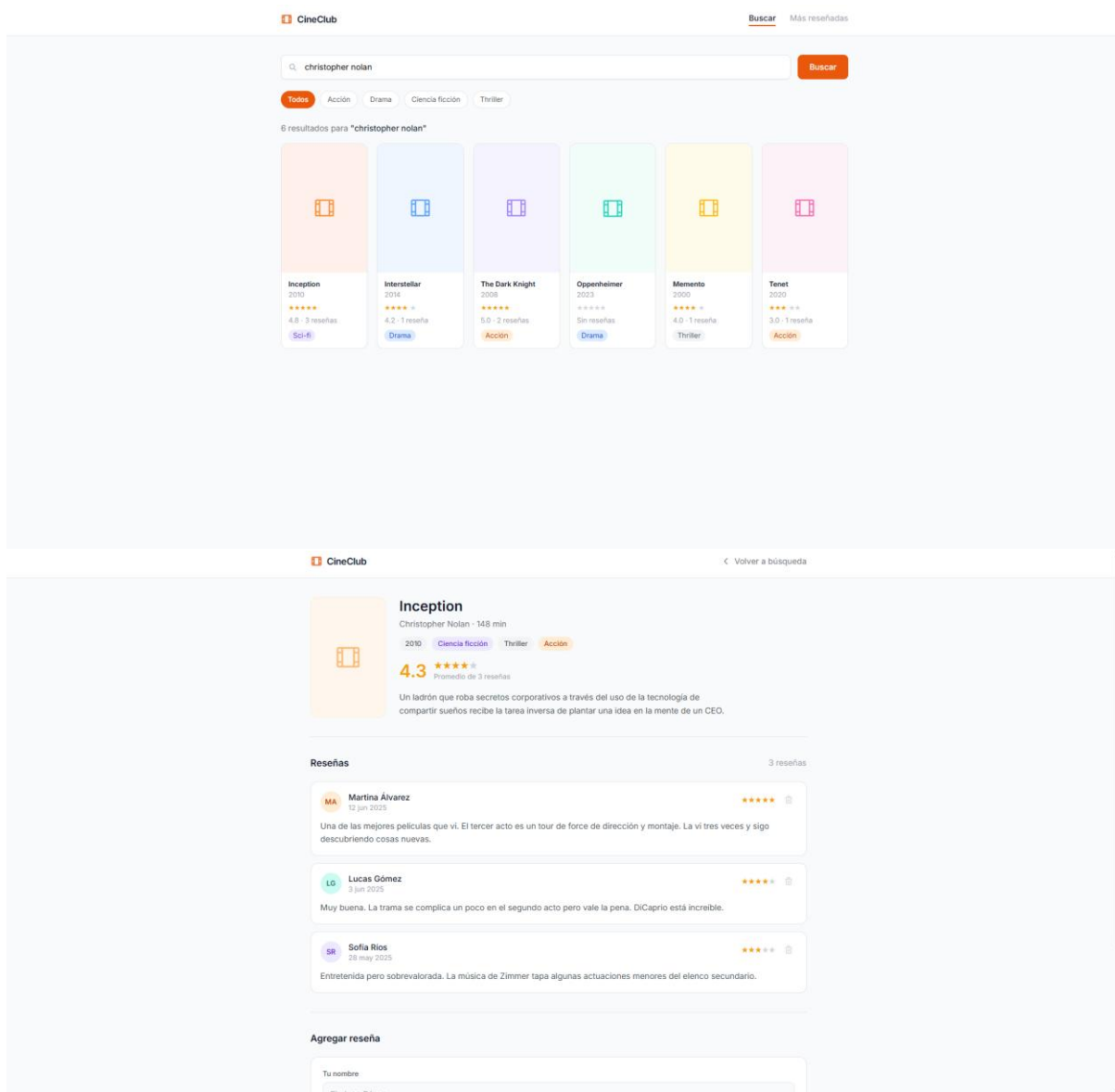
Parcial 1: CineClub

Objetivo

Construir una plataforma web fullstack donde los usuarios puedan buscar películas reales usando la API publica de TMDb (The Movie Database) y escribir reseñas propias con puntaje del 1 al 5.

El frontend en React se comunica exclusivamente con el backend propio. Es el servidor Express quien consulta a TMDb, combina esos datos con las reseñas guardadas en memoria y devuelve la respuesta al cliente.

Las reseñas se guardan en un array en memoria del servidor. Al reiniciar la aplicación se pierden.



Flujo de Datos



El frontend nunca llama a TMDB directamente. Todo el tráfico pasa por el servidor propio.

Configuración inicial: TMDB

Para acceder a la API de películas es necesario obtener una API key gratuita:

- Crear una cuenta en <https://www.themoviedb.org>
- Ir a Configuración > API y solicitar una clave de tipo Developer
- Guardar la clave en un archivo .env en la raíz del backend
- Agregar .env al .gitignore antes del primer commit

ATENCION: Las credenciales expuestas en repositorios públicos son un error grave de seguridad.

API que debe exponer el backend

Películas — proxy hacia TMDB

Método	Ruta	Descripción
GET	/api/movies/search?q=:query	Busca películas en TMDB y devuelve los resultados. Requiere el parámetro q.
GET	/api/movies/:tmdbId	Detalle de una película desde TMDB + lista de reseñas propias + avgScore calculado.

Resenas — datos propios en memoria

Metodo	Ruta	Descripcion
POST	/api/movies/:tmdbId/reviews	Agrega una reseña a una película. Campos requeridos: author, score (1-5), comment.
DELETE	/api/reviews/:reviewId	Elimina una reseña por su ID.

Requisitos técnicos

Backend — Node.js + Express (35 puntos)

- La API key de TMDB se lee desde `process.env.TMDB_API_KEY`. Nunca hardcodeada en el código.
- El archivo `.env` está en `.gitignore` y no se sube al repositorio.
- Middleware de logging para todas las requests (propio o con Morgan).
- Validación en POST `/reviews`: `author`, `score` y `comment` son obligatorios. Responder 400 si falta alguno.
- El score debe ser un numero entre 1 y 5. Responder 400 si el valor es invalido.
- Responder 404 si TMDB no encuentra la película solicitada.
- Las reseñas se guardan en un array en memoria con un id único generado con `Date.now()` o similar.
- El servidor corre en el puerto definido por `process.env.PORT` con fallback a 3001.

Frontend — React (35 puntos)

- Vista 1 — Búsqueda: input de texto, botón buscar, grilla de resultados con poster, titulo, año y `avgScore`.
- Vista 2 — Detalle: datos de la película, lista de reseñas con autor/puntaje/comentario, formulario para nueva reseña.
- La navegación entre vistas se maneja con `useState`. No se requiere React Router (es Parte 7 del curso), pero se acepta.
- El componente de búsqueda no dispara una request por cada tecla. Usar un botón de buscar o implementar `debounce`.
- Estado de carga visible mientras se espera respuesta del servidor.
- Mensaje de error visible si el servidor no responde o devuelve un error.
- Validación básica en el formulario de reseña antes de enviar: no permitir campos vacíos.
- Componentes separados con responsabilidades claras: `SearchBar`, `MovieGrid`, `MovieCard`, `MovieDetail`, `ReviewList`, `ReviewForm`.
- La URL base del backend se configura vía variable de entorno (`REACT_APP_API_URL` o `VITE_API_URL`).

Entregables y proceso (30 puntos)

- Repositorio en GitHub con historial de commits progresivo. No se acepta un único commit final con todo el código.
- `README.md` con instrucciones claras para instalar dependencias y correr backend y frontend por separado, y donde configurar la API key.
- Presentación oral: demo en vivo de la aplicación funcionando y explicación del flujo completo de una búsqueda desde el input hasta TMDB y de vuelta al browser.